Phone: (+86)18610822831
Email: jl.li@connect.ust.hk

Jialiang Li

Homepage: 0x0addc001.github.io
LinkedIn: 0x0addc001
GitHub: 0x0addc001

Education

Master of Science in Artificial Intelligence **Sep 2025 - Jan 2027 (expected)**
The Hong Kong University of Science and Technology

Main courses: Machine Learning Algorithm (Machine Learning, Deep Learning, Reinforcement Learning), Machine Learning Application (Computer Vision, Natural Language Processing, Large Language Models), Machine Learning System (Training System, Inference System), Machine Learning Ethics

Bachelor of Engineering in Computer Science **Sep 2021 - Jun 2025**
Beijing Forestry University

Main courses: Math (Calculus, Linear Algebra, Probability and Statistics, Discrete Mathematics), Coding (C Programming, C++ Programming, Java Programming, Python Programming), Electronic and Computer Engineering (Analog Circuits, Digital Circuits), Computer Science (Data Structures, Algorithm Design, Computer Organization, Operating Systems, Computer Networks, Computer Architecture, Database Systems, Compiler Principles, Computer Network Security), Software Engineering (Software Engineering, Web Front-End Development, Java Web Back-End Development, Android Mobile Development), Artificial Intelligence (Artificial Intelligence)

Work Experience

Machine Learning Engineer Intern **Jul 2026 - Oct 2026**
Shanghai Xiaomi Technology | Code-model post-training

- **Data:** Built RTL post-training data with GLM5.3-generated specifications, reasoning traces, and Random/UVM/Cocotb testbenches. Added extraction, length filtering, normalization, deduplication, self-equivalence verification, and port alignment to construct SFT and RLVR datasets.
- **Training:** Ran rejection-sampled SFT (full-parameter and LoRA) on Qwen2.5-Coder-7B, then RLVR with DAPO using executable code verification as the reward. Compared binary and linear rewards to identify gaps between reward growth, downstream correctness, and out-of-domain regression.
- **Evaluation and delivery:** Built business Benchmarks and evaluated open-source, post-trained, and commercial models with pass@k on non-agentic and agentic RTL generation. Decoupled GPU training from VCS simulation and delivered the adapted GLM5.3-744B-A40B data, training path, and smoke-test validation.

Machine Learning Engineer Intern **Jan 2026 - Apr 2026**
Hong Kong Generative AI R&D Center | Speech-model post-training

- **Data:** Synthesized trilingual Mandarin-Simplified Chinese, Cantonese-Traditional Chinese, and English annotations with Qwen3-ASR, and built an annotation-review platform. Constructed SFT and RL datasets using VAD speech ratio, SNR, speaker-independent splits, normalization, deduplication, and duration filtering.
- **Training:** Adapted Qwen3-ASR-1.7B with full-parameter SFT and GRPO reinforcement learning, tuning the training recipe for multilingual government-meeting transcription.
- **Evaluation:** Built a business Benchmark and compared open-source, post-trained, and commercial ASR models by CER. Traced language confusion, low-SNR failures, and long-audio segmentation errors back into data filtering and training settings.

Selected Projects

VideoAgent: Agent-Native Video Production Workspace **Oct 2026 - Present**

- **Agent orchestration:** Built a bounded LangGraph storyboard planner with Pydantic-validated plans and durable Agent Hook traces. Kept planning isolated from billable tools while persisting context, memory, planning, validation, and reply state.
- **Production system:** Implemented editable Settings/Outlines/Scripts, Characters/Scenes/Shots, versioned assets, view/edit collaboration, four approval gates, credit reservation, and idempotent jobs. The capability router covers 16 Pippit/Kling video models plus Auto.
- **Delivery and verification:** Used Celery and Redis for durable polling, FFmpeg for captioned multi-format final cuts, and multimodal Final Cut review for character, style, prompt, and temporal consistency. Verified the full flow with a Mock Provider, pytest, Vitest, and Playwright. Project page

LMPool: KV-Aware Routing and NVLink Transfer **Apr 2026 - Jul 2026**

- **Architecture:** Split the runtime into a launcher, one global control plane, and one data plane per GPU. The control plane tracks versioned block tables, capacity, reservations, and placement metadata while each worker retains physical ownership of its KV cache.
- **Routing and transfer:** The global scheduler combines missing prefill, queue/decode work, reclaim pressure, and free capacity. Cross-GPU moves use a prepare-execute-publish-finalize transaction with source hash/generation checks, destination reservations, and publication-before-visibility.
- **Results:** On six RTX 3090 GPUs with five repeated runs, 5x shared-prefix routing improved Qwen3-0.6B/1.7B throughput by 11.8%/21.3% and reduced mean TTFT by 18.6%/25.2% versus round robin. Load skew completed three placements and copied 87 blocks; the full system, tests, and benchmark suite are open source. Project page

Skills

Languages	Chinese (native), English (TOEFL 92)
Programming	Python, C++, C, Java, SQL, Git, Linux, Docker
Training	PyTorch (FSDP), DeepSpeed (ZeRO), Megatron (TP/PP/DP/CP/EP), PEFT (LoRA/QLoRA), verl, SFT, RFT, RLHF, RLVR, DAPO, GRPO, NCCL
Inference	vLLM, SGLang, PagedAttention, Prefix Caching, KV Cache, NCCL/NVLink, CUDA
Agent / Harness	LangChain, LangGraph, MCP, Claude Code, Codex, DeepSeek API, FastAPI
App / Delivery	Next.js, React, TypeScript, Celery, Redis, PostgreSQL/SQLite, FFmpeg
Evaluation	pass@k, CER, business benchmarks; VCS, Icarus, UVM, Cocotb; pytest, Vitest, Playwright
Release / Ops	GitHub Actions, Docker, canary release, rollback, service-health monitoring